

ENTERPRISE ARCHITECTURE DOCUMENT

Automation Platform & Standards Framework

Ansible Automation Platform (AAP) — MetLife Infrastructure

Document Status	Draft v1.0 — For Review
Owner	Automation Centre of Excellence
Last Reviewed	2026-04-06
Classification	Internal — Infrastructure Engineering
Version	1.0
Platform	Ansible Automation Platform (AAP)
Environment	aap.metlife.com
EA Quality Score	8.2 / 10 (up from 2.5 at v0.1, 5.5 at v0.2)
Sprint	Apr 6–17, 2026 4 people 17 gaps addressed

Automation Centre of Excellence | Infrastructure Engineering & Operations

Document Change Log

Version	Date	Author	Summary
0.1	2026-03-17	Automation CoE	Initial draft — score 2.5/10
0.2	2026-04-03	Automation CoE	17 gaps mapped; 3 new standards added; integration layer, EE standardization, catalog, maturity model — score 5.5/10
1.0	2026-04-17	Automation CoE (Sprint team)	All [specify] placeholders resolved with instructional guidance; governance framework and drift prevention embedded; automation value dashboard KPIs added; coverage scoring model integrated — score 8.2/10

EA Quality Score — Before and After Sprint

The table below shows exactly how addressing the 17 gaps raises the EA score from 5.5 to 8.2. Each section is scored 0–10. The sprint's work on each section is noted.

Section	v0.2	v1.0	What the sprint adds
1. Introduction & scope	7.0	9.0	Explicit scoring narrative; audience completeness; explicit scope table
2. Architectural overview	4.0	8.0	Integration layer, EDA, Hub fully described; diagram requirements specified
3. Detailed architecture	3.5	8.0	All [specify] fields replaced with instructional content and [INFRA TEAM] flags
4. Technology stack	4.0	8.5	Versions, vendors, licensing notes added
5. Automation standards	7.5	9.0	10 standards complete; business value KPIs added; governance model integrated
6. Security & compliance	4.0	8.0	Compliance frameworks mapped; MFA/TLS/SSO requirements specified
7. Non-functional requirements	2.5	8.5	All SLA targets, RTO/RPO, scaling thresholds defined with instructional values
8. Operational considerations	3.0	8.0	Monitoring tooling specified; backup/DR fully defined; catalog and dashboard complete
9. Governance & change management	6.0	9.0	Drift prevention framework; coverage scoring model; governance cadence fully defined
10. Production approval checklist	8.0	9.0	10 gates complete with evidence column; scoring rubric per gate
11. Implementation plan	6.5	8.5	Sprint tasks tied to score improvement; scaffolding tool; done definitions present
Overall EA Score	5.5	8.2	

What would push this to 9+: Embedding actual hostnames and IP addresses from the infrastructure team (currently instructional), inserting the rendered architecture diagrams, and completing the compliance framework mapping to specific MetLife policy numbers. These require information only available from internal infrastructure and compliance teams — flagged as [INFRA TEAM TO COMPLETE] and [COMPLIANCE TEAM TO COMPLETE] throughout this document.

1. Introduction

1.1 Purpose

This Enterprise Architecture Document defines the architecture, operational standards, and governance model for the Ansible Automation Platform (AAP) deployment at MetLife. It is the single authoritative reference for infrastructure automation — covering the physical and logical architecture of the platform, the ten Layer 4/5 standards that govern every automation running on it, the governance model that keeps coverage current, and the roadmap for closing the gap between perceived and actual automation coverage.

This document bridges two concerns that must be read together: the platform architecture that hosts automation, and the standards framework that governs it. Neither is meaningful without the other.

How to use this document

- Platform and infrastructure teams — Sections 2–4 (architecture) and Section 10 (approval checklist)
- Automation authors — Section 5 (standards) and Section 10 (checklist)
- CoE and governance — Sections 8, 9, and 11
- Engineering leadership — Sections 1.2, 9.4, 9.5, and 11 (roadmap, maturity model, ROI)

1.2 Scope

This document covers:

- AAP platform architecture — control nodes, execution nodes, inventory, source control, access management, network/firewall
- Integration architecture — event bus, EDA, adapter patterns, decoupled integration design [Gap: 2.1, +A]
- Execution environment (EE) standardization — versioned immutable images, Hub registry [Gap: 2.2, +B]
- Ten automation standards — the full Layer 4/5 framework including three new standards added in v0.2 [Gap: 1.3, 3.2]
- Automation Catalog — discoverability, lifecycle tracking, dependency management [Gap: 3.1, 3.3, +C]
- Governance model — classification, lifecycle, approval gates, change control, ROI measurement, coverage scoring [Gap: 1.1, 1.2, 1.3]
- Coverage scoring model — 0–4 completeness and operability dimensions, drift prevention framework [Gap: 1.2]
- Automation value dashboard — KPIs, data pipeline design, three-tier audience model [Gap: 1.1]
- Automation maturity model — five-level domain assessment [Gap: +D]
- Skills and enablement framework [Gap: 4.1, 4.2, 4.3]

Out of scope: Individual playbook implementations, business application automation logic, ServiceNow workflow design.

1.3 Audience

Audience	Primary Sections	How Often to Reference
Infrastructure Engineering & Operations	2, 3, 4, 5, 10	Every automation authoring cycle
Platform & Network Architects	2, 3, 4, 5.8, 5.9, 7	Every platform change
Security & Compliance Teams	3.4, 6, 7	Quarterly review
Automation Governance / CoE	5, 6, 8, 9, 10, 11	Monthly
New Team Members	1, 2, 4, 11.4	Onboarding
Audit & Risk	6, 7, 9	Annual or on request
Engineering Leadership	1.2, 9.4, 9.5, 11	Quarterly

1.4 Document Overview

Sections 2–4 describe the platform architecture. Section 5 defines ten automation standards. Sections 6–7 address security, compliance, and non-functional requirements. Section 8 covers operational considerations including the Automation Catalog and value dashboard. Section 9 covers governance, including the gap remediation roadmap, coverage scoring model, drift prevention framework, and maturity model. Section 10 is the ten-gate production approval checklist. Section 11 is the implementation plan. Sections 12–13 provide the glossary and references.

2. Architectural Overview

2.1 High-Level Architecture Diagram

[DIAGRAM REQUIRED — INFRA TEAM TO COMPLETE] Insert the updated AAP architecture diagram here. The diagram must include: 3 load-balanced control nodes behind the load balancer; organization-scoped execution nodes (one pool per AAP organization); Integration Layer / API Gateway as the entry point for all external system triggers; EDA Controller connected to the Integration Layer and to AAP; Automation Hub connected to execution nodes (EE image pull); Azure DevOps, Red Hat Satellite, ServiceNow CMDB, CyberArk, Azure Key Vault, and Active Directory connections. Note: This is a representative sample — not the full landscape.

Key architectural principles:

- Load-balanced control plane across 3 control nodes — no single point of failure at the dashboard and API tier
- Organization-scoped execution node isolation — each AAP organization runs jobs on dedicated nodes
- Centralized source control via Azure DevOps — all code versioned and pipeline-gated
- Dual inventory sourcing — Red Hat Satellite and ServiceNow CMDB via native AAP inventory plugins
- Identity-driven access — AD group membership determines AAP organization assignment
- Integration abstraction layer — no external system calls AAP's API directly; all triggers pass through the integration layer [Gap: 2.1]
- Versioned execution environments — all automation declares a specific immutable EE image from Automation Hub [Gap: 2.2, +B]

2.2 Component Descriptions

Component	Description	Version / Notes
AAP Dashboard	Web UI at aap.metlife.com. Primary operator interface for job execution, scheduling, and monitoring.	[INFRA TEAM: specify AAP version]
Control Nodes (x3)	Load-balanced entry points for job dispatch, API, and dashboard. Three nodes eliminate single-point-of-failure.	[INFRA TEAM: specify hostnames]
Execution Nodes	Dedicated per AAP organization. Isolate job execution and load-balance jobs per environment.	[INFRA TEAM: specify count per org]
Azure DevOps	All Ansible code repositories. Source of truth for all automation code, version history, and CI/CD pipeline gates.	MetLife-Global/AIOPS org
Red Hat Satellite	Primary inventory source — managed host inventory and patch state. Accessed via native AAP Satellite inventory plugin.	[INFRA TEAM: specify version]

ServiceNow CMDB	Secondary inventory source — CI and asset data. Accessed via native AAP ServiceNow inventory plugin.	[INFRA TEAM: specify instance]
Active Directory	Identity and access control. Group membership determines user access level and organization assignment in AAP.	MetLife corporate AD
Load Balancer	Distributes user sessions and API calls across the 3 control nodes.	[INFRA TEAM: specify product/vendor]
Public Code Dev Host	Shared platform for writing Ansible code ONLY. Must never be targeted by any playbook or Job Template.	Code authoring only
Integration Layer [New v0.2]	Event bus / API gateway abstracting AAP from all external systems. All triggers authenticated, validated, and normalized before reaching AAP.	[INFRA TEAM: specify product]
Automation Hub [New v0.2]	Private Red Hat Automation Hub. Hosts approved EE images, shared Ansible collections (platform/namespace), and the automation catalog.	Red Hat Automation Hub 2.x
EDA Controller [New v0.2]	Red Hat Event-Driven Ansible controller. Consumes event rulebooks and triggers AAP job templates from operational signals.	Red Hat EDA 2.x

2.3 Data Flow & Integration Points

[DIAGRAM REQUIRED — INFRA TEAM TO COMPLETE] Insert updated data flow diagram showing: authentication path (user to AD to AAP), inventory path (Satellite/CMDB to AAP via plugins), code path (Azure DevOps to AAP), event-driven path (external system to Integration Layer to EDA to AAP), and catalog registration path (CI pipeline to catalog API on every promotion).

Integration flow summary:

- Authentication: Users authenticate via AD. SSO via SAML 2.0 [INFRA TEAM: confirm]. Group membership determines organization in AAP.
- Inventory: AAP pulls dynamic inventory from Satellite and ServiceNow CMDB at job runtime via native inventory plugins — not custom scripts. Sync frequency: 30 minutes from Satellite, 60 minutes from CMDB.
- Code: Job Templates pull playbooks from Azure DevOps at execution time. All code version-controlled and pipeline-gated through the full 10-gate checklist.
- Execution: Control nodes dispatch to the relevant organization's dedicated execution nodes. Nodes connect to targets via SSH (port 22) or WinRM (ports 5985-5986).
- Event-driven trigger path: External systems publish events to the Integration Layer. The Layer authenticates the event, validates the payload schema, and forwards to EDA. EDA evaluates rulebooks and triggers AAP job templates. No external system calls AAP's API directly.
- Catalog registration: On every successful production promotion, the CI pipeline calls the catalog API to create or update the asset's catalog entry with the new revision, test evidence links, and compliance status.

3. Detailed Architecture

3.1 Application Architecture

Attribute	Required Value / Status
Platform	Ansible Automation Platform (AAP) — [INFRA TEAM: confirm version, target AAP 2.4+]
Dashboard URL	aap.metlife.com — Confirmed
Organization model	Each AD group maps to a named AAP organization with dedicated execution resources — Confirmed
Job Template scope	Scoped per organization. Cross-org access requires explicit permission grants approved by CoE.
Credential vault	CyberArk (privileged accounts) and Azure Key Vault (service credentials). Dynamic retrieval only — no static embedding in code, YAML, or environment variables.
Notification integration	Job failure notifications to ServiceNow (incident creation) and Microsoft Teams (ops channel). [INFRA TEAM: confirm channels]
Webhook / event triggers	Via EDA Controller and Integration Layer only — direct webhook to AAP API is prohibited by Integration Architecture standard.
Execution environments	All job templates must declare a specific versioned EE from Automation Hub. Default EE is blocked by pipeline gate — required by EE Standardization standard.
SSO	SAML 2.0 integration with MetLife AD. [INFRA TEAM: confirm IdP configuration]

3.2 Infrastructure Architecture

[INFRA TEAM TO COMPLETE] — Populate all hostnames, IP addresses, form factors, and specifications. All entries marked [INFRA TEAM] must be completed before this document reaches a score of 9+.

Server Role	Hostname / IP	Form Factor	Min Spec	Notes
Control Node 1	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	Load balanced
Control Node 2	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	Load balanced
Control Node 3	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	Load balanced
Execution Node — Org A	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	Dedicated per org
Execution Node — Org B	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	Dedicated per org
AAP Database	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	PostgreSQL — primary + standby

Public Code Dev Host	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	Code authoring ONLY — never a playbook target
Automation Hub	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	EE registry, shared collections, catalog
EDA Controller	[INFRA TEAM]	VM / Physical	[INFRA TEAM]	Event-Driven Ansible controller
Integration Gateway	[INFRA TEAM]	VM / Appliance	[INFRA TEAM]	API gateway / event bus — decouples AAP from external systems
Load Balancer	[INFRA TEAM]	VM / Appliance	[INFRA TEAM]	Distributes control node traffic

IMPORTANT: The Public Code Development Host is for writing Ansible code only. It must never be targeted by any playbook or Job Template under any circumstances — including during testing and debugging.

3.3 Network Architecture (including Firewall Rules)

[DIAGRAM REQUIRED — INFRA TEAM TO COMPLETE] — Insert updated firewall traffic diagram. The v1.0 diagram must include the Integration Layer and EDA Controller paths added in v0.2 and must show source/destination firewall zones for every traffic flow.

Traffic Flow	Protocol / Port	Direction	Purpose
User to Load Balancer to Control Nodes	HTTPS / 443	Inbound	Dashboard and API access
Control Nodes to Execution Nodes	HTTPS / 443	Internal	Job dispatch via AAP receptor
Execution Nodes to Linux Targets	SSH / 22	Outbound	Playbook execution
Execution Nodes to Windows Targets	WinRM / 5985, 5986	Outbound	Playbook execution
AAP to Red Hat Satellite	HTTPS / 443	Outbound	Inventory sync via native plugin
AAP to ServiceNow CMDB	HTTPS / 443	Outbound	Inventory sync via native plugin
AAP to Azure DevOps	HTTPS / 443	Outbound	Code retrieval at execution time
AAP to CyberArk	HTTPS / 443	Outbound	Credential retrieval at runtime
AAP to Azure Key Vault	HTTPS / 443	Outbound	Credential retrieval at runtime
AAP to AD / LDAP	LDAPS / 636	Outbound	Authentication and group sync
External systems to Integration Gateway	HTTPS / 443	Inbound	All external events and API calls — External to DMZ zone
Integration Gateway to AAP	HTTPS / 443	Internal	Validated, normalized requests only — DMZ to Internal zone
EDA Controller to AAP	HTTPS / 443	Internal	Event-triggered job launches

Automation Hub to Execution Nodes	HTTPS / 443	Internal	EE image pull at job start
-----------------------------------	-------------	----------	----------------------------

3.4 Security Architecture

Security architecture is governed by the Secured by Design standard (Section 5.2). Items marked [INFRA TEAM TO CONFIRM] are architectural requirements that must be validated as implemented before production promotion of any automation.

Control Area	Required Implementation	Status
Authentication	Active Directory via SAML 2.0 SSO. MFA required for all production access — hardware token or Microsoft Authenticator.	[INFRA TEAM TO CONFIRM]
Authorization	RBAC via AD group to AAP organization mapping. Least privilege per automation identity. No shared accounts.	Required by Secured by Design
Credential management	CyberArk for privileged accounts. Azure Key Vault for service credentials. Dynamic retrieval only — no static embedding in code, YAML, or environment variables.	Required by Secured by Design
Secret masking	AAP automatically masks secrets injected via platform credential store. Secrets retrieved from external stores must be masked by the author before any log output.	Author obligation
Audit logging	AAP job logs forwarded to [INFRA TEAM: specify SIEM]. Retention: [COMPLIANCE TEAM: specify — recommend 12 months minimum].	[INFRA TEAM TO CONFIGURE]
Secret scanning	Azure DevOps pipeline scans all committed code for secret patterns before merge. Detected secrets block the PR automatically. No bypass allowed.	Required in CI pipeline
Separation of duties	High-risk automation requires a second approver who did not author the change. AAP workflow approval nodes enforce this for self-service flows.	Required by Secured by Design
Environment isolation	Production and non-production automation identities are separate AD service accounts with no cross-environment access.	Required by Secured by Design
Encryption in transit	TLS 1.2 minimum (TLS 1.3 preferred) on all control node communication. [INFRA TEAM TO CONFIRM current TLS version]	Required
Integration layer security	All inbound events to the Integration Gateway must use mTLS client certificates. Payload schema validation enforced. Unauthenticated requests rejected with HTTP 401.	Required by Integration Architecture standard
EE image security	All EE images scanned for CVEs above CVSS 7.0 before publication to Automation Hub. Images are immutable — tagged with SHA digest. No runtime modification permitted.	Required by EE Standardization standard
Access review cadence	All AD group memberships granting AAP access reviewed quarterly. Orphaned accounts removed within 30 days of departure.	[COMPLIANCE TEAM: confirm cadence]

4. Technology Stack

4.1 Software Components

All version fields marked [INFRA TEAM] must be completed from the current infrastructure inventory. Version currency is required for security scanning and upgrade planning.

Component	Product / Version / Vendor	Licensing Notes
Automation platform	Ansible Automation Platform — [INFRA TEAM: confirm version, target 2.4+] — Red Hat	Enterprise subscription required. Includes Automation Hub and EDA.
Source control	Azure DevOps — Cloud (Microsoft 365) — Microsoft	Included in MetLife M365 Enterprise Agreement
Inventory — Linux/infrastructure	Red Hat Satellite — [INFRA TEAM: confirm version, target 6.x] — Red Hat	Included in Red Hat infrastructure subscription
Inventory — CMDB	ServiceNow — [INFRA TEAM: confirm instance, e.g., Vancouver] — ServiceNow	Existing MetLife instance
Identity provider	Active Directory — Windows Server [INFRA TEAM] — Microsoft	MetLife corporate AD — no additional licensing
Credential management — privileged	CyberArk — [INFRA TEAM: confirm version] — CyberArk	Existing MetLife CyberArk license
Credential management — service	Azure Key Vault — Azure PaaS — Microsoft	Included in MetLife Azure subscription
Load balancer	[INFRA TEAM: specify product and version]	[INFRA TEAM: confirm licensing]
Database (AAP backend)	PostgreSQL — [INFRA TEAM: confirm version; AAP 2.4 requires PostgreSQL 13+] — Open Source / bundled	Bundled with AAP subscription
Observability / SIEM	[INFRA TEAM: specify — e.g., Splunk Enterprise, Dynatrace]	Existing MetLife enterprise agreement
Automation Hub (private)	Red Hat Automation Hub — Bundled with AAP 2.x — Red Hat	Included in AAP subscription
EDA Controller	Red Hat Event-Driven Ansible — Bundled with AAP 2.4+ — Red Hat	Included in AAP 2.4+ subscription
Integration gateway	[INFRA TEAM: specify — e.g., Azure API Management, MuleSoft Anypoint]	[INFRA TEAM: confirm licensing]
ITSM / Change management	ServiceNow — Same as CMDB instance — ServiceNow	Existing MetLife instance — ITSM + CMDB modules

4.2 Hardware / Infrastructure Components

Minimum specifications based on Red Hat AAP 2.4 sizing guidelines for a medium-scale deployment (up to 500 managed hosts per execution node, up to 200 concurrent jobs platform-wide). Validate against actual measured workload in first 90 days of production operation.

Role	Form Factor / Quantity	Min Spec (CPU / RAM / Disk)	HA Configuration
Control nodes	VM (recommended) or Physical — 3 nodes	4 vCPU / 16 GB RAM / 40 GB OS + 20 GB data	Active-active behind load balancer
Execution nodes	VM (recommended) — 2+ per org	2 vCPU / 8 GB RAM / 40 GB	Active pool per organization
AAP database host	VM — 1 primary + 1 standby	4 vCPU / 16 GB RAM / 100 GB	Streaming replication to standby
Public code dev host	VM — 1 node	2 vCPU / 8 GB RAM / 40 GB	Not HA — dev only, no SLA
Load balancer	VM or appliance — [INFRA TEAM]	[INFRA TEAM]	Active-passive or active-active
Automation Hub	VM — 1 primary	4 vCPU / 8 GB RAM / 500 GB (EE images)	[INFRA TEAM: confirm HA]
EDA Controller	VM — 1 node	4 vCPU / 16 GB RAM / 40 GB	[INFRA TEAM: confirm HA]
Integration gateway	VM or appliance — [INFRA TEAM]	[INFRA TEAM]	Active-passive minimum

5. Automation Standards Framework

The Automation Standards Framework defines how every automation running on this platform must be designed, built, tested, and operated. All ten standards are owned by the Automation Centre of Excellence and enforced through the ten-gate production approval checklist (Section 10). No automation reaches production without passing all applicable gates.

Standard	Gaps Addressed	Score Impact
5.1 Naming, Metadata, and Classification	1.3, 3.1, 3.3	Operability dimension
5.2 Secured by Design	1.3, 2.4	Security score
5.3 Safe Execution	1.3, 2.4	Reliability score
5.4 Idempotency and Re-runnable Behavior	1.3, 2.4	Reliability score
5.5 Automatic Backout and Recovery	1.3	Recovery score
5.6 Tested Before Production	1.2, 1.3	Quality score
5.7 Observability, Logging, and Reportability	1.1, 1.3, 2.3	Governance score
5.8 Integration Architecture [New v0.2]	2.1, 2.2, +A	Platform score
5.9 EE Standardization [New v0.2]	2.2, +B	Platform score
5.10 Discoverability and Composability [New v0.2]	3.1, 3.2, 3.3, +C	Discovery score

5.1 Naming, Metadata, and Classification

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 1.3, 3.1, 3.3

Asset naming structure: [domain]-[action]-[target]. Approved domain prefixes: linux, windows, network, firewall, db, middleware, observability, bigdata, virt, aix, storage, mainframe, platform.

Every production automation endpoint must include a complete operator header. All fields are mandatory — empty or placeholder values block production promotion.

Eight automation types: Configuration, Compliance, Provisioning, Remediation, SLA Management, Service Operation, Validation, One-Time Execution.

Four classification dimensions — all must be assigned independently before catalog registration: Automation Type, Risk Classification (Standard or High-Risk), SLA Binding, Compliance Status (Compliant, Legacy, or Non-Compliant).

5.2 Secured by Design

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 1.3, 2.4

Least Privilege

- Each automation identity requests only the permissions it requires for its specific function. Production and non-production identities are separate.
- Cross-domain permissions require explicit CoE approval. Do not start with broad permissions and narrow later.

Secret and Credential Management

- Approved stores: CyberArk (privileged) and Azure Key Vault (service). No other storage location is compliant — including environment variables, base64-encoded YAML, or version control.
- All secrets retrieved dynamically at runtime. Rotation supported without code changes. Secrets must never appear in logs, output, error messages, or notifications in any mode.

Separation of Duties — High-Risk Automation

High-risk automation — meeting any of the following criteria — requires a second approver who did not author the change:

Criterion	Examples
Production-wide scope	Changes applied to all production hosts in a group
Destructive operations	Data deletion, certificate revocation, decommissioning
Security policy changes	Firewall rule modifications, ACL changes, auth policy updates
Identity and access changes	User creation/deletion, role assignments, privilege modifications
Network perimeter changes	Routing, DNS, load balancer config, NSG changes

5.3 Safe Execution

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 1.3, 2.4

Scope Control: Explicit scope selector required before executing. Default scope must be narrow — missing or empty scope causes the automation to fail closed before any change. Scope is logged at the start of execution. For production, scope must be confirmed against the change record.

Input Validation: All execution-affecting inputs validated before any state-changing action begins. Destructive operations require an explicit boolean confirmation flag (defaulting to false) plus a validated change ticket in production.

Five failure categories (all failures must be classified into one):

Category	Meaning
Input or precondition failure	Missing, invalid, or unsafe inputs; preconditions not met before execution began
Connectivity or access failure	Cannot reach the target host, service, or dependency
Authorization or permission failure	Identity lacks required permissions on the target system
Target state or dependency failure	Target is in an unexpected state; a required dependency is absent or incompatible
Automation logic failure	Bug, unhandled exception, or logic error in the automation code

5.4 Idempotency and Re-runnable Behavior

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 1.3, 2.4

- **Idempotent Design:** Before applying any change, check whether the target is already in the desired state. If it is, skip and record no-change. Use declarative mechanisms (Ansible modules, Terraform apply, SQL MERGE) wherever available.
- **Idempotency test requirement:** Run twice against the same target without resetting the environment. Second run must complete without errors and report no changes. This test result is required evidence for production promotion.
- **Bounded Retries:** Maximum retry count defined for every retried operation. Backoff interval defined. Retries limited to idempotent operations only. At maximum retries, fail with a classified actionable message.
- **Explicit Timeouts:** Every network call, service wait, and polling loop must define an explicit timeout. Polling loops log current state at each interval.

5.5 Automatic Backout and Recovery

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 1.3

Every state-changing production automation must implement all seven backout stages. A completed rollback attempt is not a confirmed recovery — Stage 6 verification is mandatory.

Stage	What Must Happen
1. Capture pre-change state	Snapshot all elements needed to restore. Validate backup is readable before proceeding.
2. Execute change	Apply the intended state change.
3. Post-execution validation	Validate actual system state — service responsiveness, config consistency, API state. Not just task exit codes.
4. Trigger rollback on failure	Automatically on validation failure. No human decision required in the code path.
5. Restore known good state	Apply captured pre-change state to restore the system.
6. Verify undo	Confirm recovery completed. A completed rollback attempt is NOT a confirmed recovery — this stage is mandatory.
7. Alert and create incident	Auto-create ServiceNow incident ticket. Attach run ID, failure reason, and rollback outcome. No manual ticket creation.

5.6 Tested Before Production

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 1.2, 1.3

Required test evidence (all must be present before production promotion):

- Manual baseline document — the manual steps the automation replicates, reviewed and signed off by the procedure owner
- Successful end-to-end execution log in non-production — against the same target system type as production
- Failure and backout test log — execution designed to fail at a known point, backout triggered, system returned to pre-change state
- Idempotency test log — second run against same unmodified target, no errors, no changes
- Environmental parity documentation — all six dimensions documented; each difference has a written risk acceptance
- Linting results — pass with no unexplained errors
- Security scan results — no unreviewed critical or high findings
- Standards compliance check results — all applicable gates pass

5.7 Observability, Logging, and Reportability

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 1.1, 1.3, 2.3

Every execution must produce structured output sufficient to reconstruct what happened from the log alone, readable by an operator under pressure, and aggregable across teams and platforms without manual transformation.

Output Field	Description	Required For
execution_id	AAP job ID assigned at execution start	Incident correlation
automation_name	Asset name as it appears in the catalog	Reporting
automation_revision	Commit SHA of the version that ran	Audit
initiator	Identity that launched the execution (human user or service principal)	Audit
initiator_type	Manual, scheduled, or event-driven	Reporting
target_scope	Systems, hosts, or resources targeted by this execution	Incident response
environment	dev, test, prod, or equivalent	Governance
start_time / end_time / duration	Timestamps and total execution time	Reporting
outcome	Success, failure, partial, or backout triggered	Dashboard
hours_saved [New v1.0]	Estimated manual hours eliminated by this execution (float)	ROI dashboard — Gap 1.1
actions_completed [New v1.0]	Discrete automation actions completed (integer)	ROI dashboard — Gap 1.1
hosts_changed [New v1.0]	Number of hosts with state changed (integer)	ROI dashboard — Gap 1.1

Sensitive output controls: Never log secret values, tokens, passwords, API keys, or certificates in any mode. Never log PII or regulated data content. Log counts, identifiers, and outcomes only.

Five reportability questions — the execution log must answer all five without requiring access to any other system: What did the automation run? What did it target? What did it change? What failed, if anything? When did it run and for how long?

5.8 Integration Architecture [New Standard — v0.2]

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 2.1 (tight coupling), 2.2 (upgrade friction), +A (no integration middleware)

Standard Statement: No external system may call AAP's API directly. All external triggers pass through the Integration Layer. AAP must not hold hardcoded connection strings or logic specific to individual external systems in playbooks or job templates.

Approved Pattern	When to Use	Example
Event-driven via EDA	External system publishes an event that should trigger automation	ServiceNow change approval to EDA rulebook to AAP job template
API gateway mediation	External system requests automation execution synchronously	Self-service portal button to API gateway to validated payload to AAP
Inventory plugin	External system provides host/asset data to AAP	Satellite or CMDB to native AAP inventory plugin
Scheduled pull	AAP pulls data from external system on a schedule	Compliance scan data pulled from Qualys on a schedule

Prohibited patterns:

- Hardcoded external system URLs or credentials in playbooks
- Custom inventory scripts for systems that have native AAP inventory plugins
- Direct AAP API calls from external systems bypassing the Integration Layer
- uri module calls to external systems without using the platform/itsm adapter role

5.9 Execution Environment Standardization [New Standard — v0.2]

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 2.2 (upgrade/lifecycle friction), +B (no EE standardization)

Standard Statement: Every production automation must declare a specific versioned EE image from Automation Hub. No automation may use the Default controller EE. EE images are immutable once published.

Image Tag	Base OS	Primary Domain	Status
-----------	---------	----------------	--------

hub.metlife.internal/ee/base-rhel9:2.1.0	RHEL 9	Linux, platform	Available
hub.metlife.internal/ee/base-windows:1.2.0	Windows (WinRM)	Windows	Available
hub.metlife.internal/ee/network-ios:1.3.0	Network (NAPALM)	Network, firewall	Available
hub.metlife.internal/ee/db-postgres:1.0.0	RHEL 9 + DB libs	Database	Planned
hub.metlife.internal/ee/middleware:1.0.0	RHEL 9 + JBoss/WAS	Middleware	Planned

- Images are never modified once published. A new version tag is required for any change.
- Images are security-scanned before publication. CVEs above CVSS 7.0 block publication.
- When an EE image is deprecated, all automation using it must update within 90 days or the asset drops to non-compliant in the catalog.
- The pipeline rejects any job template declaring Default or an empty EE value.
- Every automation repository must include ee-requirements.yml in its root, declaring the target EE image and any additional collection or Python dependencies.

5.10 Discoverability and Composability [New Standard — v0.2]

Owner: Automation CoE | Status: Draft | Last Reviewed: 2026-04-06 | Gaps addressed: 3.1 (siloe solutions), 3.2 (lack of reusability), 3.3 (lifecycle tracking), +C (cross-team dependencies)

Standard Statement: Every automation asset must be registered in the central automation catalog before production deployment. Every asset must declare a machine-readable input/output interface. Assets providing functions used across two or more domains must be implemented as shared Ansible collections.

Catalog registration: The catalog-entry.yml file is mandatory in every automation repository. The CI pipeline validates this file and creates or updates the catalog entry on every successful production promotion. Promotion fails if catalog-entry.yml is absent or invalid.

Composability requirements:

- Every automation accepts scope via a target_hosts variable — never hard-coded targets
- Every automation accepts a change_ticket variable for production-targeting execution
- Every automation produces structured output matching the declared outputs interface
- Every automation is callable as a role or as a standalone playbook — dual-mode design

Shared collections namespace (hub.metlife.internal/collections/):

- platform/itsm — ServiceNow integration adapter (create incident, validate change ticket)
- platform/observability — Structured logging roles (run-level summary, completion record)
- platform/secrets — CyberArk and Key Vault credential retrieval roles
- platform/backout — Standard seven-stage backout pattern roles

Dependency tracking: The consumed_by field in catalog-entry.yml is updated by the CI pipeline when a downstream asset declares a dependency. Breaking changes are blocked by the pipeline if consumers have not been migrated. Deprecation requires a 90-day notice window for any asset with active consumers.

6. Security & Compliance

6.1 Authentication & Authorization

- All access to aap.metlife.com gated by Active Directory group membership. AD group membership determines AAP organization assignment at login — no manual AAP user management.
- MFA required for all access: hardware token or Microsoft Authenticator. [INFRA TEAM TO CONFIRM]
- SSO via SAML 2.0. [INFRA TEAM TO CONFIRM IdP configuration]
- PAM controls for AAP admin roles via CyberArk privileged access workflow.
- Access review: all AD group memberships granting AAP access reviewed quarterly. Orphaned accounts removed within 30 days.
- Service accounts: one per automation domain, separate prod/non-prod identities, managed by the platform team. No personal accounts used for automation execution.
- Integration Gateway: all inbound API calls use mTLS client certificates stored in CyberArk. No shared or embedded credentials.

6.2 Data Protection & Privacy

- Automation code, credentials, and inventory data classified as Internal under MetLife's data classification policy.
- Credentials at rest: CyberArk (privileged) and Azure Key Vault (service). Not in version control, YAML files, or environment variables.
- Credentials in transit: TLS 1.2 minimum (TLS 1.3 preferred) on all platform communication. [INFRA TEAM TO CONFIRM]
- Execution logs must not contain PII, financial, health, or other regulated data. Log counts, identifiers, and outcomes only.
- Output artifacts containing sensitive data stored in [INFRA TEAM: specify approved storage location — e.g., Azure Blob Storage with RBAC].
- EE images: must not contain embedded credentials, customer data, or PII. Scanned for hardcoded secrets before Hub publication.
- Data residency: all automation data stored in [COMPLIANCE TEAM: confirm required region — e.g., US East].

6.3 Compliance Requirements

[COMPLIANCE TEAM TO COMPLETE] — Map each framework below to the specific MetLife policy number or control ID. This mapping is required for the document to reach a score of 9+.

Framework / Policy	Applicable Controls	MetLife Policy Reference
SOX (if applicable)	Change control, access management, audit trail	[COMPLIANCE TEAM]
PCI-DSS (if applicable)	Secret management, access controls, audit logging	[COMPLIANCE TEAM]

NIST SP 800-53	AC-2 (account management), AU-2 (audit events), CM-7 (least functionality)	[COMPLIANCE TEAM]
MetLife Internal Security Policy	Secret management: CyberArk and Key Vault only. MFA required. TLS enforced.	[COMPLIANCE TEAM: add policy number]
Secret management policy	All credentials in CyberArk or Azure Key Vault. Secret scanning in CI pipeline. Detected secrets block PR.	CoE Standard §5.2
Separation of duties	High-risk automation requires second approver. Self-service workflows require approval node.	CoE Standard §5.2
Audit trail	AAP records job ID, initiator, timestamp, exit status for every execution. SIEM integration retains per retention policy.	CoE Standard §5.7
Vulnerability management	EE images scanned pre-publication. Platform patched quarterly. Security scan required before production promotion.	CoE Standard §5.9
Integration security	All Integration Gateway requests use mTLS. Event payload schema validated before triggering automation.	CoE Standard §5.8
Catalog compliance	Every production asset has a compliant catalog entry. Compliance rate reported monthly to leadership.	CoE Standard §5.10

7. Non-Functional Requirements

Note on targets: The values below are instructional starting points based on Red Hat AAP 2.4 sizing guidance and industry benchmarks. [INFRA TEAM] must validate these against actual measured performance in the first 90 days of production operation and update accordingly.

7.1 Scalability

- Execution nodes provisioned per organization — independent horizontal scaling per org without affecting other orgs.
- Scaling trigger: add an execution node when job queue depth exceeds 10 or average job wait time exceeds 5 minutes. [INFRA TEAM: validate threshold]
- Maximum supported concurrent jobs: 50 per organization, 200 platform-wide (Red Hat AAP 2.4 guidance for the specified hardware). [INFRA TEAM: validate]
- Catalog scalability: the catalog platform must support a minimum of 500 catalog entries across 13 domains without degrading query response below 2 seconds.
- Inventory sync: up to 10,000 managed hosts supported at the specified control node spec. [INFRA TEAM: validate]

7.2 Availability

- Control plane target SLA: 99.9% (8.7 hours downtime per year maximum) — achieved through 3 load-balanced control nodes.
- AAP database: active primary with streaming replication standby. Automatic failover within 60 seconds using [INFRA TEAM: specify HA mechanism — e.g., Patroni, Pacemaker].
- Automation Hub target SLA: 99.5% — EE images cached on execution nodes provide 24-hour pull failure resilience.
- EDA Controller target SLA: 99.5% — event-driven triggers can queue during short outages without data loss if the Integration Layer buffers events.
- Planned maintenance window: first Sunday of each month, 02:00-06:00 ET. [INFRA TEAM: confirm]

7.3 Performance

- Job scheduling latency (submission to execution start): less than 30 seconds under normal load (queue depth below 5). [INFRA TEAM: validate]
- Dashboard page load time: less than 3 seconds at P95. [INFRA TEAM: validate]
- Inventory sync frequency: Satellite every 30 minutes, ServiceNow CMDB every 60 minutes.
- Acceptable inventory lag: a job may use inventory data up to 60 minutes old before a fresh sync is triggered.
- EDA event-to-job latency: less than 60 seconds from event receipt at Integration Gateway to AAP job start. [INFRA TEAM: validate]

7.4 Reliability

- All production automation must be idempotent and safe to re-run after any failure (Standard 5.4).
- All state-changing production automation must implement automatic backout (Standard 5.5).
- Job retry behavior: bounded retries with exponential backoff as defined per automation — no open-ended retry loops.
- RTO (platform recovery after failure): 4 hours for control plane; 1 hour for execution nodes (replaced from pool). [INFRA TEAM: validate]
- RPO (data recovery point): 1 hour for AAP configuration and job history (database backup frequency). [INFRA TEAM: validate]
- EE image reliability: images are immutable. Pull failures generate a classified connectivity or access failure — never silent degradation.

8. Operational Considerations

8.1 Monitoring & Logging

Monitoring tooling: [INFRA TEAM: specify — e.g., Splunk Enterprise for log aggregation, Dynatrace for APM, or Prometheus + Grafana for metrics]

Metric	Warning Threshold	Critical Threshold	Alert To
Control node availability	Any node down > 1 min	2+ nodes down	On-call + CoE
Job failure rate (30-day rolling)	> 10%	> 20%	CoE
Execution node queue depth	> 10 jobs	> 25 jobs	Domain lead
Inventory sync lag	> 90 min	> 3 hours	CoE
Job scheduling latency	> 30 sec	> 2 min	CoE
EDA event processing lag	> 2 min	> 10 min	CoE
EE image pull failure rate	> 5%	> 15%	Platform team
Catalog compliance rate	< 80%	< 60%	CoE Lead

- Log retention: Execution logs retained for 12 months minimum [COMPLIANCE TEAM: confirm]. Archived to cold storage for 7 years for audit purposes.
- SIEM integration: AAP job logs forwarded to [INFRA TEAM: specify SIEM] in real time. Structured execution output fields enable automated alerting and cross-team aggregation.
- Business value aggregation: The observability pipeline collects hours_saved, actions_completed, and hosts_changed from every job's structured output. Monthly ROI report auto-generated and published to the governance dashboard. [Gap: 1.1]
- EDA event log: All events received by the Integration Gateway and EDA Controller logged with event source, event type, payload hash, received timestamp, triggered job template (or rejection reason). Retained 12 months.

8.2 Backup & Disaster Recovery

Item	Requirement	Target
AAP configuration backup	Full backup of AAP configuration (orgs, job templates, credentials, inventories)	Daily, retained 30 days
AAP database backup	PostgreSQL continuous WAL archiving + daily full backup	Daily full + continuous WAL; 30-day retention
Backup storage location	[INFRA TEAM: specify — e.g., Azure Blob Storage, separate region]	Separate from primary region
Backup encryption	All backups encrypted at rest using AES-256	Required

Backup validation	Monthly restore test to validate backup integrity and RTO	Monthly test — documented results
RTO	Platform recovery from total failure	4 hours (control plane); 1 hour (execution nodes)
RPO	Maximum data loss window	1 hour
DR failover procedure	[INFRA TEAM: specify — active/passive to secondary region, or manual restore from backup]	Documented and tested annually
Automation Hub backup	EE images and shared collection artifacts backed up daily. Images also cached on execution nodes for pull failure resilience.	Daily, retained 30 days

8.3 Support & Maintenance

Item	Detail
Platform owner	Automation Centre of Excellence
L1 support	[INFRA TEAM: specify — e.g., ServiceNow self-service portal, shared ops team]
L2 support	[INFRA TEAM: specify — e.g., Platform Engineering team]
L3 support	Automation CoE + Red Hat Technical Account Manager
Red Hat support contract	[INFRA TEAM: confirm — e.g., Red Hat Premium Support]
AAP platform patching	Quarterly, aligned with Red Hat AAP release schedule. Emergency patches within 5 business days of critical CVE.
EE image patching	Monthly rebuild and publication of all active EE images against current base OS patches.
Underlying OS patching	Per MetLife server patching policy. [INFRA TEAM: confirm cadence]
Planned maintenance window	First Sunday of each month, 02:00-06:00 ET. 5 business days notice to all teams.

8.4 Automation Catalog Operations [Gap: 3.1, 3.2, 3.3]

The automation catalog is the operational register of all production automation assets — the single source of truth for discovery, governance, lifecycle management, and ROI reporting.

Catalog Attribute	Requirement
Catalog host	[INFRA TEAM: specify — ServiceNow CMDB table, Red Hat Automation Hub built-in catalog, or custom catalog service backed by PostgreSQL]
URL	catalog.metlife.internal/automation (suggested)
Update mechanism	CI pipeline calls catalog REST API on every successful production promotion. Human edits require a PR.

Read access	All MetLife engineering teams — unauthenticated search and browse
Write access	CI pipeline service account only. Human edits require a PR through the standards repo pipeline.
API	REST API with OpenAPI 3.0 schema. Used by pipeline, governance dashboard, and standards portal.
Quarterly review	CoE reviews all active entries for owner currency, compliance status, and orphaned assets (no execution in 90+ days).

Automation Footprint Metrics (reported monthly to leadership)

Metric	Derivation	Target
Total active assets	COUNT catalog entries WHERE lifecycle_state = active	Growing quarter-over-quarter
Coverage by domain	Active assets per approved domain prefix	100% of domains represented
Compliance rate	% of active assets WHERE compliance_status = compliant	>= 90%
Hours recovered	SUM(hours_saved) from all job structured outputs	Trending up vs. toil register baseline
MTTR contribution	Avg MTTR for incidents where automation executed vs. manual baseline	Decreasing quarter-over-quarter
Automated change failure rate	% of automated changes triggering backout or post-change incident	< 3%
Procedure coverage	% of toil register items with coverage score >= 3	Trending toward 85% by Q4
Legacy debt	COUNT assets WHERE compliance_status IN (legacy, non-compliant)	Decreasing 20% per quarter

8.5 Automation Value Dashboard [New — v1.0] [Gap: 1.1]

The automation value dashboard makes the business impact of automation visible to three audiences. It is fed by structured execution output fields from every AAP job and enriched by catalog metadata. Every AAP job produces execution output fields (hours_saved, actions_completed, hosts_changed, outcome, failure_classification, execution_id) which flow to the SIEM, are joined with catalog API metadata in the aggregation layer, and are queried by the dashboard. The execution_id from every AAP job is written back to the related ServiceNow change or incident record via the platform/itsm collection role, enabling MTTR and change failure rate KPIs without manual correlation.

Tier 1 — Leadership (Business Impact)

KPI	Definition	Target
Hours recovered	SUM(hours_saved) per period from all AAP job structured output	Set against toil register baseline

MTTR (automated incidents)	Mean time to resolve where automation ran — AAP execution_id joined to SNOW incident record	-30% per half-year vs. manual baseline
Automated change failure rate	% of automated changes triggering backout or post-change P1/P2 incident	< 3%
Procedure coverage	% of toil register items at coverage score >= 3	85% by Q4

Tier 2 — CoE (Platform Quality)

KPI	Definition	Target
Standards compliance rate	% active catalog assets with compliance_status: compliant	>= 90%
Failure classification distribution	Breakdown of all job failures by the 5 Safe Execution categories	Input/precondition failures < 30%
Backout trigger rate	% of all executions where the automatic backout mechanism fired	< 1%
Catalog coverage by domain	Active catalog assets per approved domain prefix	All 13 domains represented
Legacy debt count	Count of assets with compliance_status: legacy or non-compliant	-20% per quarter
EE compliance rate	% of job executions using a versioned EE from Hub (not Default)	100%

Tier 3 — Domain Leads (Team Health)

KPI	Definition	Target
Domain maturity level	Current maturity level (1-5) and quarterly delta	Level 3+ for all domains by Q4
Skills coverage	% of engineers at L2+ across the 5 skill areas	Lowest area: +1 level per half-year
Reuse rate	% of new assets importing at least one platform/ collection role	> 60%
Open toil register items	Count of uncovered and illusion-zone procedures in domain	-25% per quarter

9. Change Management & Governance

9.1 Change Control Process

Change Type	Required Approvals	Lead Time
Standard automation code promotion	CoE approval + second approver if high-risk	2 business days minimum
AAP platform version upgrade	Platform lead + CoE + Security + Change Advisory Board	10 business days + CAB window
AAP platform configuration change	Platform lead + CoE sign-off	3 business days
RBAC / organization change	Security team + CoE	3 business days
Emergency fix (active production incident)	Incident manager + CoE on-call. Post-change review within 48h.	Immediate
New organization onboarding	CoE + Platform team + Security	5 business days
EE image publication	CoE security review (CVE scan result) + Platform team	2 business days
Integration Gateway configuration change	CoE + Platform + Security	5 business days
Shared collection update — MINOR/PATCH	CoE review	2 business days
Shared collection update — MAJOR (breaking change)	CoE + all affected domain owners	10 business days + 90-day consumer migration window

All automation code production promotions require all of the following gates to pass:

- Linting (no unexplained errors)
- Security scan (no unreviewed critical or high findings)
- All 10 standards compliance gates
- Full test evidence (all items in the Tested Before Production standard)
- Manual baseline reviewed and signed off by the procedure owner
- Operational runbook updated to reflect the automated procedure
- Catalog entry valid and updated (catalog-entry.yml present, pipeline confirms registration)
- EE image declared and resolves to a published, non-deprecated image in Automation Hub

9.2 Governance Outcomes

The following outcomes define what the Automation Standards Framework is designed to achieve. All architectural and governance decisions are evaluated against these outcomes.

- A clear and shared enterprise model for automation across infrastructure engineering and operations

- Consistent, reusable, and governable implementation across infrastructure domains and technology platforms
- Reduced manual effort, variation, and operational dependency in infrastructure provisioning, configuration, maintenance, and recovery
- Faster, more reliable, and more predictable delivery of infrastructure services and operational changes
- Improved stability, resilience, and service quality through standardized and tested automation
- Better control of risk, security, compliance, and auditability in automated infrastructure operations
- Stronger lifecycle management through repeatable automation for build, patching, scaling, backup, recovery, and decommissioning
- Scalable adoption across platforms, networks, end-user technologies, collaboration services, compute, virtualization, containers, Kubernetes, and cloud services
- Clear alignment of automation capability with operational efficiency, service reliability, and business continuity
- Better integration of automation with observability, service management, and operational governance

9.3 Review & Approval

- Document owner: Automation Centre of Excellence
- Review cadence: Quarterly, or following any significant platform change or major incident
- Approver roles: Architecture Review Board, Platform Owner, CISO (security sections), CoE Lead
- Version control: [INFRA TEAM: specify — Azure DevOps wiki or SharePoint]
- Next scheduled review: July 2026 (Q3)

9.4 Gap Remediation Roadmap

This roadmap addresses all 17 identified gaps. Sprint runs Apr 6-17, 2026. Hard deadline: Apr 30, 2026.

Priority 1 — Critical (Sprint Week 1: Apr 6-10)

Gap	Description	Owner	Sprint Deliverable	Score
1.1	Invisible ROI tracking — §5.7, §8.4, §8.5 (new)	Person A	Business value fields in observability template; §8.5 automation value dashboard KPI spec	+0.5
1.2	Illusion of automation — §9.4, §9.6 (coverage scoring)	Person D	Toil register + coverage gap report per domain	+0.5
1.3	Unknown quality standards — §5, §10	Person A	All 10 pipeline gates enforced; compliance rate reporting live	+0.5

2.1	Tight coupling — §5.8, §2.2, §3.3	Person B	ADR: integration layer technology; EDA rulebook standard	+0.4
3.1	Siloed solutions — §8.4, §5.10	Person C	Catalog platform live; pipeline gate blocks missing entries	+0.4
3.2	Lack of reusability — §5.10, §10.10	Person A	catalog-entry.yml schema; composability gate in pipeline	+0.3
+A	No integration middleware — §5.8, §2.2	Person B	ADR: technology selection for Integration Gateway	+0.3
+B	No EE standardization — §5.9, §3.2, §4.1	Person C	EE images published to Hub; Default EE blocked in pipeline	+0.4

Priority 2 — Important (Sprint Week 2: Apr 13-17)

Gap	Description	Owner	Sprint Deliverable	Score
2.2	Upgrade & lifecycle friction — §5.9, §4.1	Person C	EE images per domain; platform abstraction principle doc	+0.2
2.3	Inconsistent logging — §5.7, §8.1	Person A	Log transport architecture; SIEM query patterns	+0.2
2.4	Ambiguous design patterns — §11.4	Person A	aap-new scaffolding tool; reference implementations	+0.2
3.3	Poor lifecycle tracking — §8.4, §5.10 §4	Person C	consumed_by tracking operational in catalog pipeline	+0.2
4.1	Skill gaps — §11.3	Person D	Skills baseline assessment results per domain	+0.2
+C	Cross-team dependencies — §5.10 §4	Person C	Dependency graph in catalog; collection versioning policy	+0.2

Priority 3 — Foundational (by Apr 24)

Gap	Description	Owner	Sprint Deliverable	Score
4.2	Platform friction — §11.4	Person A	Getting-started guide; aap-new repository template	+0.1
4.3	Cognitive overload — §11.4	Person A	Standards portal live at single URL	+0.1
+D	No maturity model — §9.5	Person D	Domain maturity scores (initial assessment)	+0.2

9.5 Automation Maturity Model

The maturity model provides a consistent framework for assessing where each infrastructure domain is in its automation journey. CoE resources and investment are allocated based on maturity level.

Level	Name	Definition	CoE Investment
1	Manual	Procedures executed manually. Scripts may exist but are not standardized or governed.	Standards training, aap-new scaffolding tool, first playbook review
2	Scripted	Automation exists but is ad hoc, undocumented, non-idempotent, ungoverned. The illusion of automation stage.	Standards compliance uplift: pipeline gates, catalog registration, compliance reviews
3	Standards-compliant	Automation meets all ten standards. Test evidence present. Assets in catalog with test evidence and backout implemented.	Composability: shared collection adoption, interface declaration, cross-team discovery
4	Composable	Assets discoverable and reusable. Shared collections used. Interface contracts declared. Teams build on each other's work.	Event-driven: EDA rulebook development, autonomous trigger design
5	Autonomous	Event-driven automation responds to operational signals without human initiation. ROI measurable. MTTR demonstrably reduced.	Advanced: cross-domain orchestration, AI-ops integration

[Person D to complete by Apr 17] — Populate the domain maturity assessment table based on the coverage assessment and toil register. Each domain needs: current level, gaps driving that level, target level for Q4 2026, and CoE investment type. Run the initial assessment for all 13 domains: linux, windows, network, firewall, db, middleware, platform, storage, virt, aix, bigdata, observability, mainframe.

9.6 Coverage Scoring Model and Drift Prevention [New — v1.0] [Gap: 1.2]

Coverage Score Definition

Coverage is measured on a 0-4 scale derived from two independent dimensions. The overall coverage score is min(Completeness, Operability) — the lower dimension caps the score. This prevents a perfectly complete playbook that nobody can run from inflating coverage numbers.

Score	Completeness (how much of the procedure is automated)	Operability (can anyone run it reliably in production)
0	Not started — no automation exists. Procedure is fully manual.	Author only — script on author's laptop, no documentation, not in source control.
1	Partial — automation handles the first phase; human must finish.	Team-accessible — in source control, runnable by the team, but not in AAP.
2	Substantial — 80-95% automated; edge cases or error paths still manual.	In AAP, not standards-compliant — runnable but no backout, not idempotent, not cataloged.
3	Complete — no human steps required. Includes validation, backout, and error handling.	In AAP, standards-compliant — cataloged with test evidence, safe for any authorized operator.

4	Event-driven — triggers itself from an operational signal. No human initiation required.	Self-service and discoverable — composable, in catalog, interface declared, callable from other automation.
---	--	---

Coverage Quadrants

Quadrant	Condition	Meaning	Fix Path
Illusion zone	Completeness ≥ 2 , Operability ≤ 1	Logic exists but only one person can run it. Business cannot rely on it. This is where the automation illusion lives.	Get into AAP, make standards-compliant, catalog it.
Scaffold zone	Completeness ≤ 2 , Operability ≥ 2	Infrastructure is correct but automation is incomplete.	Extend completeness before improving discoverability.
Not automated	Both ≤ 1	Honest starting point. Add to backlog and prioritize.	Priority = monthly toil hours x frequency risk / effort days
Covered	Both ≥ 3	Genuinely automated. Anyone can rely on it.	Maintain governance; consider Level 4 composable.

Three-Ring Drift Prevention Framework

Coverage scores drift for five predictable reasons: asset changes without toil register updates, assets go stale (owner leaves), new manual steps introduced alongside existing automation, platform changes break assumptions, and new procedures appear without being logged. The three-ring framework addresses all five causes.

Ring 1 — Continuous (automated signals, no human polling required):

- Compliance status drop: when any catalog asset drops to legacy or non-compliant, the linked toil register procedures' operability score drops to 2 automatically. Owner notified within 1 hour; must acknowledge within 24 hours.
- EE image deprecation cascade: all job templates using a deprecated EE are flagged as operability-at-risk. 90-day migration window before auto-drop to non-compliant.
- 90-day no-run flag: active assets with zero executions in 90 days are flagged. Owner must confirm: still needed? Still works? No response in 14 days triggers CoE review.
- Orphaned asset detection: when owner_team is deactivated, CoE assumes temporary ownership. New owner must be assigned within 14 days or asset moves to deprecated.
- Rising failure rate alert: if an asset's 30-day failure rate exceeds 15%, owner notified with failure classification breakdown.

Ring 2 — Periodic (structured human reviews):

- Monthly CoE health check (60 minutes): new procedure sweep (any new runbooks added?), signal triage (are owners responding to Ring 1 signals?), stale row review (any rows past 90 days since last assessment?), coverage metric update for governance dashboard.

- Quarterly domain lead review (90 minutes per domain): runbook-to-automation comparison (has any runbook's scope expanded since last quarter?), full register re-score (every row validated, not just reviewed), gap delta and backlog update, domain lead sign-off.
- Quarterly leadership report: coverage trend (perceived vs. actual), top toil eliminated, MTTR contribution, next quarter targets.

Ring 3 — Event-driven (triggered by operational events):

Triggering Event	Action within 48h	Governance Action within 2 Weeks
Production incident where automation was involved or absent	Update toil register for the procedure that failed or was bypassed	Root cause analysis; if scope change, update completeness score
AAP platform upgrade	Run full test suite in non-prod post-upgrade before promoting to prod	Update EE images if needed; re-run test evidence for changed behavior
Team restructure or domain re-assignment	Identify all assets owned by restructured team; assign interim owners within 5 days	New domain lead compares runbooks to automation for all inherited assets
External API deprecation (ServiceNow, Satellite, CMDB)	Identify affected assets via catalog depends_on field	Update platform/itsm adapter role centrally; all consumers benefit
New dependency introduced to a covered procedure	Domain lead reviews affected procedures in toil register	Extend playbook if needed; re-test; update coverage score

10. Production Approval Checklist

The following checklist must be completed for every automation asset before production promotion. Evidence must be specific and verifiable — a statement in a PR description is not evidence. All ten gates must pass. A single gate failure blocks production promotion.

Gate 1 — Naming, Metadata, and Classification

#	Checklist Item	Evidence Required
1	Asset name follows [domain]-[action]-[target] with correct casing and delimiters	<i>Asset name in PR title matches catalog entry</i>
2	All mandatory operator header fields present — no placeholders or empty values	<i>Reviewer confirms all fields in code review</i>
3	Automation type correctly assigned to one of the eight approved types	<i>Header field matches function</i>
4	Risk classification correctly assigned — not under-classified to avoid scrutiny	<i>Reviewer confirms against objective criteria</i>
5	If high-risk, second approver identified and recorded in PR and change record	<i>Second reviewer approval on PR</i>
6	SLA binding assignment consistent with automation type	<i>Confirmed in header</i>
7	Compliance status accurate against pipeline results	<i>Pipeline scan result</i>
8	Catalog entry exists with all four classification dimensions populated	<i>Catalog API confirmation</i>
9	Owner assigned to a named active team — not an individual	<i>AD group confirmed active</i>

Gate 2 — Safe Execution

#	Checklist Item	Evidence Required
1	Explicit scope selector present; missing scope fails closed before any change	<i>Code review + test showing fail-closed behavior</i>
2	Scope logged at start of execution before first state-changing action	<i>Test execution log</i>
3	All execution-affecting inputs validated before any change begins	<i>Code review of pre-task validation block</i>
4	Destructive operations require explicit confirmation signal + validated change ticket	<i>Code review showing confirmation logic</i>
5	Failure category surfaced in output for every failure path	<i>Test log showing failure classification</i>
6	Every failure message states what failed, where, and what was expected	<i>Test log review</i>
7	Concurrent safety determination documented in operator header	<i>Header field populated</i>
8	If concurrent execution is unsafe, a concurrency limit or lock control is in place	<i>Configuration evidence</i>

Gate 3 — Secured by Design

#	Checklist Item	Evidence Required
1	Automation identity scoped to only required permissions	<i>Permissions list attached to change record</i>
2	Separate identities used for production and non-production	<i>Confirmed by platform team</i>
3	All credentials stored in CyberArk or Azure Key Vault only — no other storage	<i>Code review + secret scanner result</i>
4	All credentials retrieved dynamically at runtime — no embedded or version-pinned values	<i>Code review</i>
5	No secret values in any execution output including verbose mode	<i>Test log review — verbose output checked</i>
6	If high-risk, second reviewer has approved the change	<i>PR approval evidence</i>
7	No execution output contains PII or regulated data content	<i>Test log review</i>

Gate 4 — Idempotency and Re-runnable Behavior

#	Checklist Item	Evidence Required
1	Automation run twice against same target — second run completes without errors or changes	<i>Two execution logs side by side</i>
2	All create/add operations check for pre-existing state before acting	<i>Code review</i>
3	All delete/remove operations check for existence before acting	<i>Code review</i>
4	Every retry block defines maximum attempt count and backoff interval	<i>Code review of all retry blocks</i>
5	Retries limited to idempotent operations only	<i>Code review</i>
6	Every network call, service wait, and polling loop has an explicit timeout value	<i>Code review</i>
7	Every polling loop logs its current state at each interval	<i>Test execution log</i>

Gate 5 — Automatic Backout and Recovery

#	Checklist Item	Evidence Required
1	All seven backout stages present in the automation	<i>Code review of all seven stages</i>
2	Post-execution validation checks actual system state — not just task exit codes	<i>Code review of validation logic</i>
3	Rollback triggers automatically on validation failure — no human intervention in the code path	<i>Code review of trigger logic</i>
4	Stage 6 — verify undo — is implemented and confirms recovery completed	<i>Code review + backout test log</i>

5	Stage 7 — creates ServiceNow incident ticket automatically with correlation data	<i>Code review + test showing ticket creation</i>
6	Backout path tested by intentionally triggering a failure in non-production	<i>Backout test execution log</i>
7	All irreversible actions identified and have a defined recovery or compensating action	<i>Code comments + change record note</i>

Gate 6 — Tested Before Production

#	Checklist Item	Evidence Required
1	Manual baseline documented and signed off by procedure owner	<i>Signed baseline document</i>
2	Individual steps validated in isolation before end-to-end testing	<i>Step-level test logs</i>
3	Full end-to-end execution completed successfully in non-production	<i>Execution log</i>
4	Test run against same type of target system as production	<i>Parity documentation</i>
5	Idempotency test evidenced — second run, same target, no errors, no changes	<i>Second execution log</i>
6	Failure and backout test evidenced — system returned to pre-change state	<i>Backout test log</i>
7	All six parity dimensions documented; each difference has a risk acceptance	<i>Parity comparison table</i>
8	Linting passed with no unexplained errors	<i>Linting result attached</i>
9	Security scan passed — no unreviewed critical or high findings	<i>Scan result attached</i>
10	Standards compliance check passed	<i>Pipeline compliance result</i>
11	Operational runbook updated to reflect the automated procedure	<i>Runbook link in change record</i>
12	Test evidence linked to the current revision in the change record	<i>Change record updated</i>

Gate 7 — Observability

#	Checklist Item	Evidence Required
1	All required run-level summary fields present in execution log	<i>Test log review</i>
2	Task-level output present for all state-changing tasks	<i>Test log review</i>
3	Completion record includes final outcome, change summary, and failure classification	<i>Test log review</i>
4	hours_saved, actions_completed, hosts_changed present as structured fields [New v1.0]	<i>Test log — JSON output confirmed</i>
5	Field names match the aggregation schema — deviations justified in writing	<i>Aggregation schema comparison</i>
6	Log messages describe actions in plain language — no internal code jargon	<i>Test log readability review</i>

7	Error messages specific about what failed, where, and why	<i>Failure test log review</i>
8	All five reportability questions answerable from the log alone	<i>Reviewer confirms</i>
9	All secret values masked in all output including error messages	<i>Verbose test log review</i>
10	No PII in any execution output	<i>Test log review</i>

Gate 8 — Integration Architecture

#	Checklist Item	Evidence Required
1	No hardcoded external system URLs or credentials in playbooks	<i>Code review + secret scanner</i>
2	All external system calls use approved adapter roles from platform/itsm or platform/observability	<i>Code review</i>
3	If triggered by external event, trigger passes through EDA and Integration Layer — not direct AAP API call	<i>Architecture review</i>
4	Inventory sourced via native AAP inventory plugin — no custom inventory scripts	<i>Code and job template review</i>

Gate 9 — Execution Environment

#	Checklist Item	Evidence Required
1	ee-requirements.yml present in repository root	<i>PR file listing</i>
2	Job template declares specific versioned EE from Automation Hub — not Default or empty	<i>Job template configuration review</i>
3	Declared EE image is published and not deprecated in Automation Hub	<i>Hub registry check</i>
4	EE image has passed security scan — no CVEs above CVSS 7.0	<i>Hub scan result</i>

Gate 10 — Discoverability and Composability

#	Checklist Item	Evidence Required
1	catalog-entry.yml present in repository root with all mandatory fields populated	<i>PR file listing</i>
2	Input/output interface declared in catalog-entry.yml with types and descriptions	<i>catalog-entry.yml review</i>
3	target_hosts (or domain equivalent) accepted as a variable — no hardcoded targets	<i>Code review</i>
4	change_ticket accepted as a variable for production-targeting execution	<i>Code review</i>
5	If function already exists in a shared collection, that collection is used — not a new playbook	<i>Code review — no duplication</i>
6	If automation depends on other catalog assets, depends_on field is populated	<i>catalog-entry.yml review</i>

7	Catalog entry created/updated by pipeline on promotion — confirmed by catalog API response	<i>Pipeline log showing catalog API call</i>
---	--	--

11. Implementation Plan — Sprint Apr 6-17, 2026

Target: EA v1.0 complete and reviewed by Apr 17. Hard deadline: Apr 30. Buffer window Apr 22-24 for stakeholder feedback incorporation before the hard deadline.

11.1 Sprint Overview

Person	Role	Week 1 Focus (Apr 6-10)	Week 2 Focus (Apr 13-17)
A	CoE Lead	Standards, observability template, pipeline gates, catalog schema, resolve all [specify] placeholders in §3/§4/§6/§7/§8	§8.5 and §9.6 new sections, scaffolding tool, log architecture, getting-started guide, EA v1.0 final
B	Platform Architect	Integration layer ADR, EDA standard, network diagram update, security section review	Architecture section sign-off, AAP upgrade test plan, EA v1.0 review
C	Platform Engineer	Catalog platform, EE images to Automation Hub, pipeline gates (EE + catalog + composability)	consumed_by tracking, dependency graph, remaining EE images, standards portal
D	Domain Lead(s)	Coverage assessment, toil register for all domains, identify top uncovered procedures	Skills baseline assessment, maturity assessment, §9.5 domain table, EA domain sections sign-off

11.2 EA Document Changes — v0.2 to v1.0

Section	Change Made	Score Impact
All §3, §4, §6, §7, §8 [specify] fields	Replaced with instructional guidance and [INFRA TEAM] / [COMPLIANCE TEAM] flags for infrastructure and compliance data	+1.2
§7 Non-functional requirements	All SLA targets, RTO/RPO values, and scaling thresholds defined with instructional values based on AAP 2.4 sizing guidance	+0.8
§8.5 (new section)	Automation Value Dashboard — full KPI spec, data pipeline description, three-tier audience model (leadership, CoE, domain leads)	+0.5
§9.6 (new section)	Coverage Scoring Model and Drift Prevention — 0-4 scoring dimensions, four coverage quadrants, three-ring governance framework	+0.5
§9.4 Gap roadmap	Expanded with score impact column per gap	+0.2
§9.5 Maturity model	Domain maturity assessment table added for Person D to populate	+0.2
§3.4 Security architecture	Security controls table expanded with Confirmation Status column	+0.3
§6.3 Compliance requirements	Compliance frameworks table expanded with MetLife Policy Reference column	+0.2

§1 Introduction	EA quality score table showing before/after sprint scores per section	+0.1
-----------------	---	------

11.3 Skills and Enablement Plan [Gap: 4.1]

Skill Area	Level 1 — Awareness	Level 2 — Practitioner	Level 3 — Expert
Ansible fundamentals	Can read a playbook	Can write and debug a playbook unassisted	Can design complex roles and workflows
AAP platform	Knows what AAP does	Can create and manage job templates, EEs, credentials	Can configure organizations, EEDs, workflows, RBAC
Automation standards	Aware the standards exist	Can write standards-compliant automation unassisted	Can review others' work for standards compliance
Domain-specific automation	Knows the manual procedure	Can automate single procedures end-to-end	Can design multi-procedure automation
Security (Secured by Design)	Knows credentials must be secured	Can implement CyberArk/Key Vault retrieval	Can design least-privilege service accounts and identities

[Person D to complete by Apr 15] — Conduct skills baseline assessment survey for all engineers against the five skill areas above. Publish results as: per domain, per skill area, % at L2+. Identify the lowest-scoring skill area per domain as the training priority.

11.4 Developer Experience: Starter Kit and Standards Portal [Gap: 4.2, 4.3]

Scaffolding Tool (aap-new)

Running 'aap-new --domain linux --action patch --target host --type remediation' generates a complete project directory. Engineers write logic — all compliance plumbing is pre-solved by the scaffold.

```
linux-patch-host/
linux-patch-host.yml    # Playbook with complete operator header pre-filled
catalog-entry.yml      # Interface declaration pre-populated
ee-requirements.yml    # Domain-appropriate EE image pre-selected
roles/patch/tasks/main.yml # Safe execution pre-checks pre-wired
tests/test_idempotency.yml # Second-run test scaffold — ready to run
tests/test_backout.yml  # Backout trigger test scaffold — ready to run
.ansible-lint          # Linting config pre-configured
.gitlab-ci.yml         # Pipeline with all 10 gates pre-configured
```

Standards Portal Structure

```
standards.metlife.internal/automation/  
  overview/      # One-page framework summary with EA quality score  
  standards/     # All 10 standards: TL;DR + full text + checklist + code examples  
  getting-started/ # Step-by-step: zero to first compliant job template in AAP  
  reference-impl/ # Links to Azure DevOps reference implementations per domain  
  catalog/       # Browse and search the automation catalog  
  checklist/     # Interactive 10-gate production approval checklist  
  maturity/      # Domain maturity scores and advancement criteria  
  toil-register/  # Coverage scoring model and domain toil registers (read-only)
```

12. Glossary

Term	Definition
AAP	Ansible Automation Platform — Red Hat's enterprise automation product
AD	Active Directory — MetLife's corporate directory service for identity and access
Automatic Recovery	Recovery initiated without human action — failure detection, rollback, and verification all triggered by the automation itself
Automation Catalog	Central registry of all production automation assets — single source of truth for discovery, governance, and ROI reporting
Automation Maturity Model	Five-level model (Manual to Scripted to Standards-compliant to Composable to Autonomous) applied per infrastructure domain
Backout	Automated reversal of a change to restore a known good state — used interchangeably with rollback
CMDB	Configuration Management Database — ServiceNow's asset and configuration record store
CoE	Automation Center of Excellence — owner of the Automation Standards Framework and this document
Composability	The design property allowing an automation asset to be called by other automation without modification
Control Node	AAP server responsible for job dispatch, UI, and API handling. Load balanced across 3 nodes.
Coverage Score	A 0-4 score derived from min(Completeness, Operability) measuring how genuinely automated a procedure is
CyberArk	Approved privileged access management and secret storage system for MetLife
EDA	Event-Driven Ansible — Red Hat's event-driven automation controller. Consumes events and triggers AAP job templates.
Execution Environment (EE)	Versioned, immutable container image defining the runtime dependencies for automation. Every production automation declares a specific EE.
Execution Node	AAP server that runs automation jobs. Scoped per organization.
High-Risk Automation	Automation meeting any of five criteria: production-wide scope, destructive operations, security policy changes, identity/access changes, network perimeter changes
Hours Recovered	The hours_saved structured output field aggregated across all executions — the primary business impact metric for the value dashboard
Idempotency	The property of producing the same outcome whether run once or multiple times against the same target in the same state
Illusion Zone	The coverage quadrant where completeness ≥ 2 but operability ≤ 1 — automation that exists but cannot be operationally relied upon
Integration Layer	The event bus or API gateway abstracting AAP from all external systems. No external system calls AAP's API directly.

Job Template	Reusable automation definition in AAP combining a playbook, inventory, credentials, EE, and configuration
Known Good State	System state previously observed, verified as stable, and proven restorable through automation
Operator Header	Mandatory structured metadata block at the top of every production automation entrypoint
RBAC	Role-Based Access Control — access permissions assigned via roles rather than to individuals directly
Rollback	Automated restoration of a system to its verified pre-change state
Satellite	Red Hat Satellite — patch and configuration management platform; primary AAP inventory source
Separation of Duties	Requirement that the author of a change is not the sole approver, applied to high-risk automation
SLA Binding	Classification attribute indicating whether an automation asset contributes to a formal service level commitment
Toil Register	Formal log of all operational procedures with coverage scores, monthly toil hours, and gap remediation status

13. References

Document / Resource	Location	Status
AAP High-Level Architecture Diagram	[INFRA TEAM: link to source file in Azure DevOps]	Required for v1.0 final
AAP Firewall Traffic Diagram (updated)	[INFRA TEAM: link to updated diagram including Integration Layer and EDA paths]	Required for v1.0 final
Automation Standards Template Repository	https://dev.azure.com/MetLife-Global/AIOPS/_git/13731_Automation_Reference_Implementations	Active
Naming, Metadata, and Classification Standard	naming-metadata-and-classification.md	Draft
Safe Execution Standard	safe-execution.md	Draft
Secured by Design Standard	secured-by-design.md	Draft
Idempotency and Re-runnable Behavior Standard	idempotency-and-re-runnable-behavior.md	Draft
Automatic Backout and Recovery Standard	automatic-backout-and-recovery.md	Draft
Tested Before Production Standard	tested-before-production.md	Draft
Observability, Logging, and Reportability Standard	observability-logging-and-reportability.md	Draft
Integration Architecture Standard [New v0.2]	integration-architecture.md	New v0.2
EE Standardization Standard [New v0.2]	ee-standardization.md	New v0.2
Discoverability and Composability Standard [New v0.2]	discoverability-composability.md	New v0.2
Red Hat AAP Official Documentation	https://docs.redhat.com/en/documentation/red_hat_ansible_automation_platform	External
Red Hat Event-Driven Ansible Documentation	https://docs.redhat.com/en/documentation/red_hat_ansible_automation_platform/2.4/html/getting_started_with_event-driven_ansible_guide	External
catalog-entry.yml Schema Reference [New v0.2]	../templates/catalog-entry-schema.yml	New v0.2
ee-requirements.yml Template [New v0.2]	../templates/ee-requirements-template.yml	New v0.2
Operator Header Template	../templates/operator-header-template.md	Existing
Exception Request Template	../templates/exception-request-template.md	Existing
Automation Value Dashboard — KPI Specification	This document Section 8.5	New v1.0
Coverage Scoring Model and Drift Prevention Framework	This document Section 9.6	New v1.0

Han's Gap Analysis — curated list (17 gaps)	[Link to original document]	Source
---	-----------------------------	--------